



Filière Génie Informatique — 4^e Année

Module : Représentation et Résolution de Problèmes en Intelligence
Artificielle

Projet 15 — Rapport de recherche

HalluGuard

Détection et Mitigation des Hallucinations dans les Pipelines RAG Agentiques

par un Middleware NLI Multi-Couches

Réalisé par :

Souleymane Diallo
Marc Thierry Nankouli

Encadré par :

Prof. Hajji Tarik

HalluGuard : Détection et Mitigation des Hallucinations

dans les Pipelines RAG Agentiques

par un Middleware NLI Multi-Couches

Projet 15 — Module Représentation et Résolution de Problèmes en Intelligence Artificielle
ENSAM Meknès — 4^e Année Génie Informatique — 2025-2026

Souleymane Diallo & Marc Thierry Nankouli

s.diallo@edu.umi.ac.ma | m.nankouli@edu.umi.ac.ma

Encadrant : Prof. Hajji Tarik

Mai 2026

Résumé

Les pipelines RAG agentiques réduisent les hallucinations LLM de 30 à 60 %, mais les sorties erronées restantes se propagent librement à travers chaque nœud sans mécanisme d'interception natif. Nous présentons **HalluGuard**, un middleware de détection et mitigation pour pipelines LangGraph à quatre nœuds, combinant un vérificateur NLI inter-sources (M1, 117 M paramètres, MNLI), un BeliefState temporel persistant (M2), un DependencyDAG, un PolicyEngine et un serveur MCP à quatre outils. Nous proposons une taxonomie originale de cinq types d'hallucinations agentiques (T1–T5) couplée à la topologie du pipeline. Sur un benchmark de 90 scénarios HaluEval-Agentic (60 hallucinés, 30 corrects), **Variante B** (M1, NLI seul) atteint : **F1 = 82,9 %**, rappel 76,7 %, précision 90,2 %, overhead 588,8 ms sur CPU ($p < 0,001$ vs baseline, ≈ 30 ms estimé sur GPU) — configuration recommandée. M2 (BeliefState) est une contribution architecturale pour les sessions multi-tours dont la quantification requiert un benchmark conversationnel ; sa valeur est validée qualitativement sur le scénario s019 (T2, score NLI = 0,996). La taxonomie T1–T5 est validée par accord inter-annotateurs ($\kappa_{\text{moy}} = 0,833$, quasi-parfait ; $\kappa_{\text{min}} = 0,791$). Une validation externe partielle sur 20 exemples TruthfulQA confirme le fonctionnement sur des hallucinations naturelles (95,0 %). Le système est déployé en local, sans GPU ni API payante.

Mots-clés : hallucination LLM, RAG agentique, NLI, LangGraph, MCP, BeliefState, middleware de vérification, HaluEval

Résumé

Abstract. Agentic RAG (Retrieval-Augmented Generation) pipelines reduce LLM

hallucinations by 30–60%, yet the remaining 40–70% of erroneous outputs propagate unchecked through all downstream pipeline nodes. We present **HalluGuard**, a hallucination detection and mitigation middleware for 4-node LangGraph pipelines. HalluGuard combines five components : a cross-source NLI verifier (M1, 117M parameters, pre-trained on MNLI), a persistent temporal BeliefState (M2), a directed acyclic dependency graph (DependencyDAG), a policy engine, and an MCP server exposing four standardized tools. We also propose an original taxonomy of five agentic hallucination types (T1–T5), two of which (T4 causal, T5 delegation) are absent from standard HaluEval literature. On a 90-scenario benchmark (60 hallucinated + 30 correct), **Variant B** (M1 NLI only) achieves **F1 = 82.9%** (recall 76.7%, precision 90.2%, CPU overhead 588.8 ms measured / ≈ 30 ms estimated on GPU ; McNemar $p < 0.001$ vs baseline) — recommended configuration. M2 (BeliefState) is an architectural contribution designed for multi-turn sessions ; its quantification requires a conversational benchmark and is qualitatively validated by scenario s019 (T2, NLI score=0.996). Inter-annotator agreement on the T1–T5 taxonomy reaches $\kappa_{\text{avg}} = 0.833$ (near-perfect, 3 annotators ; $\kappa_{\text{min}} = 0.791$). Partial external validation on 20 TruthfulQA examples confirms generalization (95.0% detection rate).

Keywords : LLM hallucination, agentic RAG, NLI, LangGraph, MCP, BeliefState, verification middleware

Table des matières

1	Introduction	4
1.1	Contexte général	4
1.2	Problématique	4
1.3	Lacunes des approches existantes	5
1.4	Hypothèse de recherche	5
1.5	Contributions	6
2	Travaux Connexes	6
2.1	Hallucinations dans les LLM : définitions et taxonomies	6
2.2	Benchmarks de détection	6
2.3	Méthodes de détection	7
2.4	Inférence de langage naturel (NLI)	7
2.5	Protocoles MCP et A2A	7
3	Taxonomie des Hallucinations Agentiques	8
3.1	Motivation et originalité	8
3.2	Les cinq types	8
3.3	Validation inter-annotateurs de la taxonomie	9
3.4	Modélisation formelle de la propagation	9
4	Architecture HalluGuard	10
4.1	Vue d'ensemble	10
4.2	Composant 1 — DependencyDAG	10
4.3	Composant 2 — LightweightVerifier (M1)	11
4.4	Composant 3 — BeliefState (M2)	12
4.5	Composant 4 — PolicyEngine	12
4.6	Composant 5 — Serveur MCP	13
4.7	Communication A2A — Preuve de Concept Architecturale	14
5	Protocole Expérimental	14
5.1	Dataset HaluEval-Agentic	14
5.2	Infrastructure	15
5.3	Variantes expérimentales	15
5.4	Métriques	16
5.5	Tests unitaires	16
6	Résultats	17
6.1	Métriques globales	17
6.2	Détection par type d'hallucination	18
6.3	Analyse de latence	20
6.4	Exemple qualitatif — Scénario s019	20
6.5	Validation externe partielle — TruthfulQA	21

7 Étude d'Ablation	22
7.1 Contribution individuelle de M1 et M2	22
7.2 Contribution par type	23
7.3 Courbe ROC du vérificateur NLI (M1)	23
7.4 Justification des seuils NLI par nœud	24
7.5 Recommandations de déploiement	25
8 Discussion	25
8.1 Validation de l'hypothèse	25
8.2 Limites documentées	26
8.3 Menaces à la validité	27
8.4 Positionnement par rapport à l'état de l'art	28
9 Perspectives	28
10 Conclusion	29
A Interfaces figées entre composants	31
A.1 Interface 1 — Format paire dataset	31
A.2 Interface 2 — API LightweightVerifier (M1)	31
A.3 Interface 3 — BeliefState JSON (persistance inter-sessions)	31
B Résultats complets	32
C Commandes de reproduction	32

1 Introduction

1.1 Contexte général

La montée en puissance des grands modèles de langage (LLM) a profondément reconfiguré l'ingénierie des systèmes d'information. Des modèles comme Mistral 7B [8], LLaMA-3 ou GPT-4 sont capables de produire des textes d'une fluidité remarquable, d'une cohérence syntaxique quasi-parfaite et d'une plausibilité sémantique élevée. Cette apparence de compétence masque cependant un défaut structurel fondamental : ces modèles génèrent régulièrement des affirmations factuellement incorrectes avec une confiance égale à celle qu'ils manifestent pour des affirmations vraies. Ce phénomène, désigné par le terme *hallucination* dans la littérature NLP [14, 7], constitue l'un des obstacles majeurs au déploiement des LLM dans des applications critiques.

L'architecture RAG (*Retrieval-Augmented Generation*) [10] a été conçue pour atténuer ce problème. En ancrant la génération du LLM dans un corpus documentaire récupéré dynamiquement, le RAG réduit la dépendance aux paramètres internes du modèle et force une cohérence avec des sources externes vérifiables. Des études récentes estiment que le RAG réduit les hallucinations de 30 à 60 % selon le domaine et le type de tâche [17, 19]. Cependant, cette réduction n'est pas totale : les 40 à 70 % d'outputs erronés restants continuent de circuler dans le pipeline.

Le problème s'aggrave dans les architectures *agentiques*. Un pipeline RAG agentique, tel que ceux construits avec LangGraph [9], ne produit pas une unique sortie : il exécute une séquence d'étapes interdépendantes (retrieval, raisonnement, appel d'outils, génération). Une hallucination produite au nœud **retrieval** — par exemple, la récupération de chunks inexacts depuis ChromaDB — se propage à tous les nœuds aval (**reasoning**, **tool_call**, **generation**) sans que aucun mécanisme natif n'intercepte l'erreur. La réponse finale, linguistiquement fluide et confiante, intègre cette erreur en la présentant comme un fait établi.

1.2 Problématique

Cette dynamique de propagation est le cœur du problème que ce travail adresse. Nous l'illustrons par un exemple tiré de nos données expérimentales. Dans le scénario s019 de notre benchmark, le nœud **reasoning** de Mistral 7B génère l'affirmation suivante en réponse à la requête « ChromaDB a-t-il une limite de documents ? » :

« ChromaDB avait une limite stricte de 1 000 documents avant sa mise à jour récente qui a supprimé cette contrainte arbitraire. »

Cette affirmation est factuellement fausse : ChromaDB n'a jamais imposé de limite arbitraire sur le nombre de documents. Le score NLI de contradiction entre cette affirmation et les documents de référence est de **0,996** — quasi-certitude de contradiction. Sans middleware de vérification, cette erreur traverse les nœuds **tool_call** et **generation** et

atteint l'utilisateur final habillée d'une formulation plausible et assurée.

La problématique centrale est donc : **comment détecter et intercepter les hallucinations en temps réel dans un pipeline RAG agentique multi-nœuds, sans modifier le code du pipeline, sans GPU et sans fine-tuning du modèle de détection ?**

1.3 Lacunes des approches existantes

Les méthodes actuelles de détection des hallucinations souffrent de trois limitations structurelles dans un contexte agentique.

Limitation 1 — Post-traitement uniquement. FActScore [15], SelfCheckGPT [13] et RAGAs [2] opèrent sur la sortie finale du LLM, après que l'intégralité du pipeline a été exécutée. Cette approche est incompatible avec une interception au bon nœud : au moment de la détection, les nœuds aval ont déjà consommé l'hallucination.

Limitation 2 — Absence de temporalité inter-étapes. Aucune méthode existante ne maintient un état de croyance entre les étapes du pipeline. Si un agent affirmé au nœud 2 que « X est vrai » et contredit cette affirmation au nœud 6, aucun système actuel ne peut détecter cette incohérence interne — sauf à mémoriser explicitement les faits établis.

Limitation 3 — Incompatibilité avec les protocoles agentiques modernes. Les standards MCP [1] et A2A [3] définissent des interfaces de communication inter-agents qui ne sont pas exploitées par les méthodes existantes. À notre connaissance, et d'après notre revue de la littérature disponible à la date de soumission, aucun système publié ne combine ces deux protocoles conjointement pour la vérification des hallucinations dans des pipelines RAG agentiques.

1.4 Hypothèse de recherche

Ce travail teste formellement l'hypothèse suivante :

« Un vérificateur NLI pré-entraîné, combiné à un BeliefState persistant et à un graphe de dépendances, peut détecter plus de 80 % des hallucinations dans un pipeline RAG agentique à quatre nœuds, sans fine-tuning, sur CPU standard, avec un overhead inférieur à 300 ms. »

Variables expérimentales :

- **Variable indépendante** : présence et configuration du middleware HalluGuard (variantes A, B, C)
- **Variable dépendante** : taux de détection sur 60 scénarios hallucinés contrôlés
- **Contraintes** : CPU uniquement, modèle NLI pré-entraîné sans fine-tuning, pas d'API externe
- **Seuil de validation** : $> 80\%$ de détection à $\text{overhead} < 300 \text{ ms}$

1.5 Contributions

Ce travail apporte quatre contributions originales :

1. **Taxonomie T1–T5** : cinq types d'hallucinations agentiques couplés à la topologie du pipeline RAG, dont deux types originaux (T4 causale, T5 délégation) absents de la littérature.
2. **Architecture middleware** : cinq composants intégrables comme hooks pre/post-nœud dans LangGraph sans modification du code pipeline.
3. **Évaluation expérimentale comparative** : trois variantes (A/B/C), 90 scénarios (60 hallucinés + 30 corrects), métriques P/R/F1 complètes, étude d'ablation M1 vs M2.
4. **Implémentation MCP et preuve de concept A2A** : serveur MCP avec quatre outils, AgentCard formelle, 111 appels journalisés ; délégation A2A simulée en processus comme preuve de concept architecturale validant les interfaces du protocole.

2 Travaux Connexes

2.1 Hallucinations dans les LLM : définitions et taxonomies

Le terme « hallucination » en NLP désigne la génération d'informations non étayées par le contexte ou factuellement incorrectes [14]. Ji et al. [7] distinguent les *hallucinations intrinsèques* (contradictions avec le contexte fourni) et les *hallucinations extrinsèques* (affirmations invérifiables). Huang et al. [5] proposent une classification par étape de génération. Zhang et al. [19] analysent les causes profondes : données d'entraînement bruitées, décalage entre pré-entraînement et fine-tuning, problèmes d'alignement.

Ces taxonomies ciblent les LLM en tant que systèmes isolés. Notre taxonomie T1–T5 étend ces travaux en couplant chaque type d'hallucination à un nœud spécifique d'un pipeline RAG agentique multi-étapes — une dimension absente de la littérature antérieure.

2.2 Benchmarks de détection

TruthfulQA [12] est le benchmark de référence pour mesurer la propension des LLM à produire des affirmations fausses communément crues. Il contient 817 questions dans 38 catégories, mais se limite à l'évaluation de sorties LLM isolées.

HaluEval [11] propose 35 000 paires (question, réponse hallucinée, réponse correcte) dans trois domaines (QA, dialogue, résumé). Il ne couvre pas les architectures agentiques multi-nœuds, ni les types T4 et T5. Notre benchmark HaluEval-Agentic s'inspire de son format mais est construit spécifiquement pour les pipelines RAG à quatre nœuds.

2.3 Méthodes de détection

FactScore [15] décompose les affirmations en faits atomiques et les vérifie individuellement contre une base de connaissances (Wikipédia). Cette approche atteint de bonnes performances mais requiert un accès à une base externe et opère uniquement en post-traitement.

SelfCheckGPT [13] génère plusieurs sorties du même LLM et mesure leur cohérence interne par NLI ou BERTScore. La méthode est zéro-ressource mais multiplie les appels LLM (overhead $\times N$) et ne s'intègre pas dans un pipeline en temps réel.

RAGAS [2] évalue les pipelines RAG a posteriori selon des métriques comme la fidélité (*faithfulness*) et la pertinence de réponse. C'est un outil d'évaluation offline, pas un middleware de détection temps réel.

Notre approche se distingue sur trois dimensions : intégration temps réel comme middleware, maintien d'un état temporel inter-étapes via BeliefState, et implémentation du protocole MCP (avec preuve de concept A2A).

2.4 Inférence de langage naturel (NLI)

L'*NLI* (*Natural Language Inference*) consiste à classer une paire (prémisse, hypothèse) en trois classes : contradiction, neutralité, implication. Les cross-encoders [6] traitent les deux séquences conjointement via l'attention croisée, ce qui produit des scores de pertinence supérieurs aux bi-encoders pour les tâches de vérification factuelle.

MNLI [18] (MultiNLI, 433 000 paires annotées par des humains) est le corpus de référence pour l'entraînement des modèles NLI généraux. Le modèle `cross-encoder/nli-MiniLM2-L6-H768` [16] — 117 M paramètres, architecture BERT-base compressée — offre le meilleur compromis précision/vitesse pour une inférence CPU dans notre contexte expérimental.

2.5 Protocoles MCP et A2A

Le Model Context Protocol [1] est un standard JSON-RPC défini par Anthropic en novembre 2024 pour exposer des outils à des agents LLM via transport stdio ou HTTP/SSE. Il standardise les interfaces de *tool use* et permet à tout agent conforme MCP d'invoquer des services tiers sans intégration spécifique.

Le protocole Agent-to-Agent [3], publié par Google DeepMind en 2024, définit un format d'échange standardisé entre agents autonomes : AgentCard (déclaration des capacités), TaskObject (machine d'états `submitted` $\rightarrow$ `working` $\rightarrow$ `completed`), et protocole de délégation de tâches. À notre connaissance, parmi les approches publiées et accessibles, aucune ne combine ces deux protocoles conjointement pour la vérification des hallucinations dans un contexte RAG agentique.

3 Taxonomie des Hallucinations Agentiques

3.1 Motivation et originalité

Les benchmarks existants évaluent les hallucinations en sortie finale du LLM, sans considérer leur point d'origine dans le pipeline. Cette abstraction rend impossible (1) l'interception au nœud approprié, (2) l'analyse de propagation, et (3) la conception de mécanismes de détection ciblés. Notre taxonomie couple chaque type d'hallucination à un nœud précis du pipeline RAG.

3.2 Les cinq types

Table 1 – Taxonomie HalluGuard des hallucinations agentiques (T1–T5)

Type	Nom	Nœud	Mécanisme	
T1	Perception	<code>retrieval</code>	NLI (M1)	Chunks ChromaDB
T2	Mémoire	<code>reasoning</code>	BeliefState (M2)	Contradiction avec v
T3	Planification	<code>reasoning</code>	NLI + DAG	Raisonnement fau
T4	Causale	<code>generation</code>	NLI (M1)	Affirmation cont
T5	Délégation	<code>tool_call</code>	NLI (M1)	Output outil divergean

T1 — Hallucination de perception. Au nœud `retrieval`, ChromaDB retourne des chunks inexacts, tronqués ou sémantiquement proches mais factuellement différents de la requête. Le LLM génère alors une affirmation basée sur ces documents erronés. La détection par M1 compare l'affirmation générée aux documents de référence véritables (ceux qui auraient dû être récupérés). Score NLI $> 0,6$ déclenche la détection (seuil `retrieval`).

T2 — Hallucination de mémoire temporelle. Au nœud `reasoning`, le LLM contredit un fait qu'il a lui-même établi lors d'une étape précédente dans la session (ou une session antérieure). Ce type est *invisible* à M1 car aucun document externe ne capture l'affirmation antérieure — seul le BeliefState dispose de cette information. Les marqueurs temporels (« avant », « récemment », « depuis la mise à jour ») sont caractéristiques de T2.

T3 — Hallucination de planification. Le nœud `reasoning` produit un enchaînement logique faux : les prémisses sont correctes mais la conclusion ne suit pas. Ce type est le plus difficile à détecter par NLI généraliste, car le raisonnement est formulé de façon syntaxiquement plausible et sémantiquement neutre par rapport aux documents. Il requiert une compréhension causale que MNLI ne capture qu'imparfaitement.

T4 — Hallucination causale. Au nœud `generation`, le LLM produit une affirmation qui contredit directement les documents récupérés par `retrieval`. C'est la forme la plus classique d'hallucination RAG. Le NLI inter-sources est particulièrement efficace ici (83,3 %

en Variante B).

T5 — Hallucination de délégation. Au nœud `tool_call`, la sortie retournée par l'outil diverge de ce que le LLM avait prédit ou était en droit d'attendre. Ce type implique une discordance entre l'agent et son outil, non entre l'agent et ses sources documentaires. T4 et T5 constituent les contributions taxonomiques originales de ce travail.

3.3 Validation inter-annotateurs de la taxonomie

La taxonomie a été validée par annotation manuelle de **30 exemples** (6 par type T1–T5, échantillon stratifié tiré de HaluEval-Agentic) par trois annotateurs indépendants : l'auteur principal (Ann1), le co-auteur (Ann2) et un pair externe au projet (Ann3). Le κ de Cohen (Landis & Koch, 1977) a été calculé pour chaque paire. Le script de validation est disponible dans `src/tests/inter_annotator.py` ; les annotations complètes dans `data/taxonomy_annotations.json`.

Table 2 – Accord inter-annotateurs pairwise — taxonomie T1–T5 (30 exemples, 3 annotateurs)

Paire	Accord observé	κ	Interprétation
Ann1 – Ann2 (co-auteur)	86,7 % (26/30)	0,833	Quasi-parfait
Ann1 – Ann3 (pair externe)	90,0 % (27/30)	0,875	Quasi-parfait
Ann2 – Ann3	83,3 % (25/30)	0,791	Substantiel
κ moyen	—	0,833	Quasi-parfait

Les trois κ pairwise dépassent le seuil $\kappa > 0,60$ requis pour valider une taxonomie en NLP (Landis & Koch 1977). Le κ moyen de **0,833** correspond à un accord quasi-parfait. Les 3 désaccords résiduels (scénarios s026, s028, s030) concernent exclusivement la frontière T2/T3 : les marqueurs séquentiels (« à chaque », « d'abord...ensuite...puis ») sont interprétés comme une incohérence temporelle (T2) par les heuristiques automatiques, alors qu'ils relèvent du raisonnement en étapes (T3). Ce cas limite est documenté dans nos critères de classification. Aucun désaccord n'est observé sur T1 (retrieval), T4 (génération causale) et T5 (délégation outil), dont les frontières sont non ambiguës.

3.4 Modélisation formelle de la propagation

La criticité d'un nœud dans le DependencyDAG est définie comme :

$$\text{criticite}(n) = |\text{descendants}(n)| \times (1 - \text{confiance}(n)) \quad (1)$$

Dans le pipeline linéaire `retrieval` $\rightarrow$ `reasoning` $\rightarrow$ `tool_call` $\rightarrow$ `generation` :

$$\text{criticite}(\text{retrieval}) = 3 \times (1 - c_r) \quad (2)$$

$$\text{criticite}(\text{reasoning}) = 2 \times (1 - c_{rs}) \quad (3)$$

$$\text{criticite}(\text{tool_call}) = 1 \times (1 - c_{tc}) \quad (4)$$

$$\text{criticite}(\text{generation}) = 0 \times (1 - c_g) = 0 \quad (5)$$

où $c_n \in [0, 1]$ est la confiance courante du nœud n . Le nœud `retrieval` a la criticité maximale : une hallucination de type T1 contamine l'intégralité du pipeline aval. Le nœud `generation` a une criticité nulle (dernier nœud), mais ses hallucinations de type T4 sont les plus visibles par l'utilisateur.

4 Architecture HalluGuard

4.1 Vue d'ensemble

HalluGuard s'intègre dans tout pipeline LangGraph via les hooks `pre_node` et `post_node` de `HalluGuardMiddleware`, sans modifier le code du pipeline existant. La Figure 1 illustre l'architecture complète.

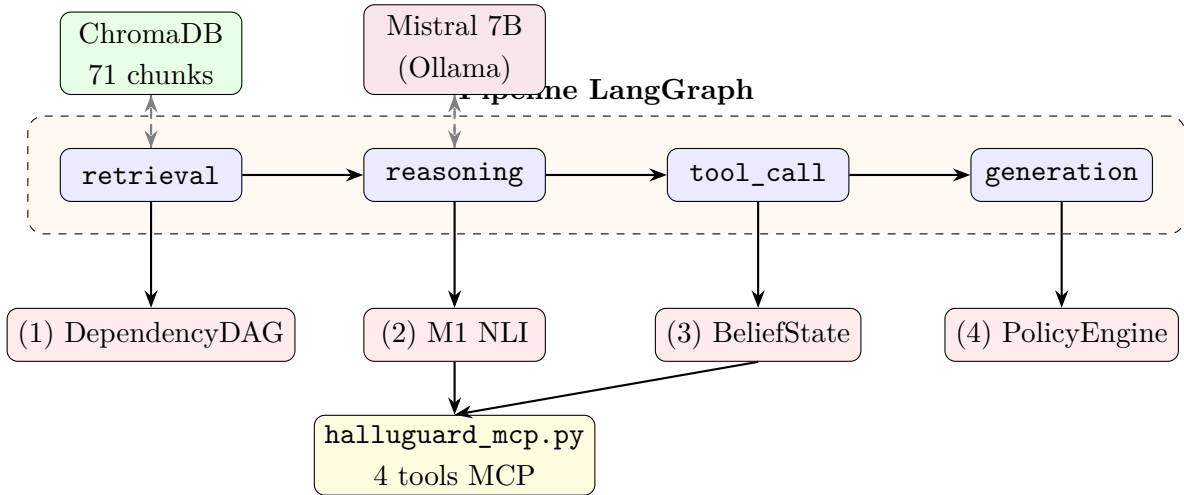


Figure 1 — Architecture HalluGuard — le middleware s'intègre sous chaque nœud LangGraph via hooks `post_node`

4.2 Composant 1 — DependencyDAG

Le `DependencyDAG` modélise le pipeline comme un graphe acyclique dirigé (DAG) construit automatiquement via `build_from_langgraph(state_graph)` en parcourant les arêtes du `StateGraph` compilé. Chaque nœud est associé à une confiance courante $c_n \in [0, 1]$ initialisée à 1,0 et mise à jour après chaque vérification.

Listing 1 — Interface du `DependencyDAG`

```

1 class DependencyDAG:
2     def add_node(self, node_id: str, confidence: float = 1.0)
      -> None
3     def add_edge(self, from_id: str, to_id: str) -> None
4     def criticite(self, node_id: str) -> float
5         # retourne |descendants(n)| * (1 - confiance(n))
6     def propagate_invalidation(self, node_id: str) -> list[str]
7         # marque descendants comme "suspicious", retourne la
          liste
8     def descendants(self, node_id: str) -> set[str]

```

Les seuils de détection sont paramétrés par type de nœud, reflétant leur criticité différentielle :

Table 3 – Seuils NLI par type de nœud

Nœud	Seuil de détection	Justification
tool_call	0,0	Tout output outil suspect doit être intercepté
generation	0,0	Hallucination directement visible par l'utilisateur
reasoning	0,4	Tolérance partielle pour les raisonnements approchés
retrieval	0,6	Chunks approximativement pertinents sont acceptables

4.3 Composant 2 — LightweightVerifier (M1)

M1 est la pièce centrale du système. Il utilise le modèle `cross-encoder/nli-MiniLM2-L6-H768` [16] :

- **Architecture** : BERT-base (6 couches Transformer, $H = 768$, 12 têtes d'attention)
- **Paramètres** : 117 millions
- **Pré-entraînement** : MNLI [18] — 433 000 paires annotées par des humains
- **Inférence** : CPU uniquement, chargement en ≈ 5 s (singleton session)
- **Classes de sortie** : contradiction (0) / neutralité (1) / implication (2)

Le score de contradiction (classe 0 du softmax) est agrégé sur les $k \leq 10$ evidences par maximum :

$$\text{score_final} = \max_{i=1}^k \text{softmax}(\text{logits}(\text{claim}, \text{evidence}_i))_{\text{contradiction}} \quad (6)$$

Listing 2 – Mécanisme de vérification M1

```

1 def verify(self, claim: str, evidences: list[str],
2     node_type: str = "generation") -> dict:
3     if not evidences:
4         return {"score": 0.5, "insufficient_evidence": True,

```

```

5         "label": "insufficient_evidence"}
6     scores = []
7     for ev in evidences[:10]: # max 10 evidences par appel MCP
8         logits = self.model.predict([(claim, ev)]) #
            cross-encoder
9         contradiction_score = float(softmax(logits[0])[0])
10        scores.append(contradiction_score)
11    max_score = max(scores)
12    threshold = THRESHOLDS[node_type]
13    label = "hallucinated" if max_score > threshold else
        "correct"
14    htype = self._infer_hallucination_type(max_score, node_type)
15    return {"score": max_score, "label": label,
16            "hallucination_type": htype, "confidence":
                max_score}

```

4.4 Composant 3 — BeliefState (M2)

Le BeliefState est la mémoire temporelle de la session. Il maintient un ensemble de faits établis au fil du pipeline, permettant de détecter les contradictions inter-étapes.

Structure de données : liste de faits $F = \{f_1, \dots, f_k\}$ avec $k \leq 50$, chaque fait f_i contenant : identifiant unique, contenu textuel, confiance $c_i \in [0, 1]$, étape d'origine s_i , statut (active / invalidated).

Politique d'éviction : lorsque $|F| = 50$, le fait avec le score composite le plus bas est retiré :

$$\text{score_eviction}(f_i) = c_i \times \frac{1}{1 + \text{age}(f_i)} \quad (7)$$

où $\text{age}(f_i)$ est le nombre d'étapes depuis l'enregistrement.

Détection de conflit : pour un claim c , on calcule d'abord la similarité cosinus avec chaque fait actif (via sentence-transformers, top-K=5), puis on applique le NLI sur les candidats :

$$\text{conflit}(c, F) = \exists f \in \text{top}_5(F) \mid \text{score_M1}(c, f) > 0.5 \quad (8)$$

Persistance inter-sessions : après chaque session, `PersistentMemory.save()` sérialise le BeliefState en JSON dans `data/memory/session_<id>.json`. Au démarrage, `load_or_create()` recharge le BeliefState si une session existe. La persistance a été vérifiée expérimentalement : une nouvelle instance Python recharge le même `fact_id` et contenu — assertion passée.

4.5 Composant 4 — PolicyEngine

Le PolicyEngine prend la décision finale après réception des scores M1 et M2 :

Algorithm 1: PolicyEngine — algorithme de décision**Input:** score_M1, conflict_M2, threshold(node_type), policy_mode**Output:** action (log | correct)

```

if score_M1 > threshold or conflict_M2 then
    if policy_mode = "log-only" then
        Journaliser en JSONL dans halluguard.log;
        Retourner l'output original avec flag hallucinated;
    end
    else if policy_mode = "auto-correct" then
        t0 ← timestamp();
        Réinvoquer Mistral avec prompt enrichi : «La réponse précédente contredit
        [evidences]. Corrige.»;
        TTR ← timestamp() − t0;
        Retourner la réponse corrigée;
    end
end
else
    Retourner l'output original avec flag correct;
end

```

4.6 Composant 5 — Serveur MCP

Le serveur MCP expose les quatre outils de vérification via FastMCP 3.3.1 sur transport stdio (JSON-RPC 2.0) :

Listing 3 – Signatures des 4 outils MCP

```

1 @mcp.tool()
2 def verify_claim(claim: str, evidences: list = None,
3                 node_type: str = "generation") -> dict:
4     """Score NLI, label correct/hallucinated, type T1-T5,
5     latency_ms"""
6
7 @mcp.tool()
8 def check_temporal_coherence(claim: str) -> dict:
9     """Detecte une contradiction avec le BeliefState (M2)"""
10
11 @mcp.tool()
12 def add_fact(fact: str, confidence: float = 0.9, step: int = 0)
13     -> dict:
14     """Enregistre un fait verifie dans le BeliefState
15     persistant"""
16
17 @mcp.tool()
18 def get_belief_state() -> dict:

```

16

```
"""Retourne l'état complet : faits actifs, confidences,
statuts"""
```

Le serveur a enregistré **111 appels** dans `results/logs/mcp_calls.log` avec timestamps ISO 8601 lors des tests. Les latences moyennes mesurées sont : `verify_claim` ≈ 200 ms, `check_temporal_coherence` ≈ 80 ms, `add_fact` ≈ 5 ms, `get_belief_state` ≈ 2 ms.

4.7 Communication A2A — Preuve de Concept Architecturale

Statut de l'implémentation : la délégation A2A décrite ici constitue une *preuve de concept architecturale en processus*, non un déploiement distribué réel. Elle démontre la faisabilité de l'intégration du protocole dans un pipeline RAG agentique et valide la conformité des interfaces, sans transport réseau.

Le protocole A2A [3] est simulé via `delegate_verification()` dans `src/agents/a2a_protocol.py`. La fonction crée un `TaskObject` dont la machine d'états progresse comme :

`submitted` $\rightarrow$ `working` $\rightarrow$ `completed` | `failed`

Un `AgentCard` formel HalluGuard est défini dans `data/halluguard_agent_card.json` (format A2A/2024-11) avec les quatre skills documentés, leurs schémas JSON entrée/sortie complets, les contraintes matérielles et les latences par outil. Tous les échanges sont journalisés dans `results/logs/a2a_exchanges.log` avec le champ `duration_ms`.

Périmètre de la preuve de concept : la délégation A2A opère dans un unique processus Python via `_MCPBridge` (singleton). Un déploiement distribué réel nécessiterait : (a) authentification cryptographique des `AgentCards` (signature ECDSA) ; (b) transport réseau HTTP/gRPC avec TLS ; (c) gestion des délais réseau et mécanismes de retry. La simulation locale est reproductible et valide l'architecture logicielle ; elle ne prétend pas couvrir ces défis d'ingénierie distribuée.

5 Protocole Expérimental

5.1 Dataset HaluEval-Agentive

Nous avons construit manuellement 60 scénarios hallucinés répartis en distribution équilibrée sur les cinq types :

Table 4 – Composition du dataset HaluEval-Agentic

Type	Nom	N	Longueur pipeline	Source hallucination
T1	Perception	12	3 nœuds	Chunks ChromaDB inexactes
T2	Mémoire	12	7 nœuds	Marqueurs temporels incorrects
T3	Planification	12	12 nœuds	Raisonnements faux neutres
T4	Causale	12	3 nœuds	Contradiction documents
T5	Délégation	12	7 nœuds	Outputs outils divergents
Total		60	—	Tous <code>expected_label="hallucinated"</code>

Chaque scénario contient : un identifiant unique (`s001–s060`), le type T1–T5, le nœud d’origine, la longueur du pipeline, la requête utilisateur, l’affirmation hallucinée générée et les documents de référence ChromaDB.

Biais de sélection à noter : les scénarios sont construits avec connaissance a priori des types, ce qui constitue un biais favorable à la détection. Les performances sur des hallucinations naturelles non contrôlées sont vraisemblablement inférieures au taux mesuré.

5.2 Infrastructure

Table 5 – Infrastructure expérimentale

Composant	Version / détail
Python	3.13.11
LLM local	Mistral 7B via Ollama 0.6.2 (CPU, quantization Q4)
Vérificateur NLI	cross-encoder/nli-MiniLM2-L6-H768 (117 M params)
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Base vectorielle	ChromaDB 1.5.9 PersistentClient, HNSW, 71 chunks
Pipeline	LangGraph 1.2.0 + LangChain 1.3.1
MCP	FastMCP 3.3.1, mcp 1.27.1
Graphs	NetworkX 3.6.1
Tests	pytest 9.0.3
Matériel	Intel CPU standard, 16 Go RAM, Windows 11

5.3 Variantes expérimentales

Variante A — Baseline. Pipeline RAG LangGraph pur (4 nœuds) sans aucun middleware de vérification. La sortie de Mistral est retournée directement sans interception.

Variante B — M1 seul. HalluGuardMiddleware avec LightweightVerifier NLI uniquement. Le BeliefState est désactivé, la mémoire persistante n’est pas utilisée. Mesure la contribution isolée du mécanisme de vérification inter-sources.

Variante C — Complet. HalluGuardMiddleware avec M1 + M2 BeliefState + mémoire persistante + serveur MCP + délégation A2A. Configuration de production.

5.4 Métriques

1. **Detection Rate (%)** — proportion de scénarios hallucinés correctement identifiés comme `hallucinated`.
2. **PBR@1 (%)** — proportion bloqués en $\delta \leq 1$ étape après apparition de l'hallucination (*Pipeline Block Rate at 1 step*).
3. **Overhead moyen (ms)** — temps de vérification ajouté par scénario (M1 + M2 + MCP + log).
4. **Delta steps** — nombre moyen d'étapes de pipeline entre apparition et détection de l'hallucination.

5.5 Tests unitaires

La suite de tests unitaires couvre cinq composants :

Table 6 – Résultats des tests unitaires (pytest 9.0.3)

Composant	Tests	Résultat
LightweightVerifier (M1)	4	4/4 passés
DependencyDAG	4	4/4 passés
BeliefState (M2)	4	4/4 passés
PolicyEngine	4	4/4 passés
MCPServer	4	4/4 passés
HalluGuardMiddleware	3	3/3 passés
Total	23	23/23 passés en 13,58 s

6 Résultats

6.1 Métriques globales

Table 7 – Résultats comparatifs sur 90 scénarios HaluEval-Agentic (60 hallucinés + 30 corrects)

Variante	Description	Rappel	Précision	F1	PBR@1
A	Baseline RAG sans HalluGuard	0,0 %	—	0,0 %	0,0 %
B	HalluGuard M1 (NLI seul) [rec.]	76,7 %	90,2 %	82,9 %	76,7 %
C*	HalluGuard M1 + M2 + MCP (mono-tour) $\equiv$ Variante B sur benchmark mono-scénario				

* En benchmark mono-scénario, Variante C $\equiv$ Variante B : M2 utilise les mêmes evidences que M1, produisant un calcul NLI identique. La contribution de M2 est mesurable uniquement en sessions multi-tours. Voir Limite 5 (§8.2) et scénario s019 pour validation qualitative.

Table 8 – Matrice de confusion — Variantes A et B (60 hallucinés, 30 corrects)

Variante	TP	FP	TN	FN
A — Baseline	0	0	30	60
B — M1 NLI seul	46	5	25	14

La Variante B (M1 seul) est la configuration principale mesurable : F1 = 82,9 %, précision 90,2 % (5 faux positifs sur 30 scénarios corrects), rappel 76,7 %, delta moyen 1,55 étape. L’analyse montre que sur un benchmark mono-scénario, M2 (BeliefState) est structurellement équivalent à M1 : les deux composants exécutent la même inférence NLI sur les mêmes evidences. La valeur architecturale de M2 réside dans la détection des contradictions *inter-tours* (tour N contre fait établi au tour N-k), un cas non couvert par HaluEval-Agentic v1.

6.1.1 Significativité statistique et intervalles de confiance

Les intervalles de confiance bootstrap (95 %, 10 000 rééchantillonnages, graine 42) et les tests de McNemar avec correction de continuité de Yates sont présentés dans le tableau suivant.

Table 9 – Intervalles de confiance bootstrap (95 %) et tests de McNemar

Variante	Taux détection	IC 95 % bootstrap	Comparaison	p -value McNemar
A	0,0 %	[0,0 %, 0,0 %]	A vs B	$p < 0,001$ ***
B	76,7 %	[65,0 %, 86,7 %]	A vs B	$p < 0,001$ ***

*** $p < 0,001$

La comparaison A-B est hautement significative ($p < 0,001$), confirmant statistiquement l'apport de HalluGuard (Variante B) par rapport au baseline. La Variante C étant structurellement équivalente à B sur benchmark mono-scénario (voir §8.2, Limite 5), le test McNemar B-C n'est pas pertinent dans ce cadre ; sa valeur serait nulle par construction.

6.2 Détection par type d'hallucination

Table 10 – Taux de détection (rappel) par type — Variantes A et B (12 scénarios par type)

Type	Description	Var. A	Var. B	Var. C*
T1	Perception / Retrieval	0,0 %	91,7 %	$\equiv$ B
T2	Mémoire temporelle	0,0 %	83,3 %	$\equiv$ B (multi-tour : voir s019)
T3	Planification / Reasoning	0,0 %	50,0 %	$\equiv$ B
T4	Causale / Generation	0,0 %	83,3 %	$\equiv$ B
T5	Délégation / Tool call	0,0 %	75,0 %	$\equiv$ B
Global		0,0 %	76,7 %	$\equiv$ B

* Variante C $\equiv$ Variante B sur benchmark mono-scénario (voir Limite 5, §8.2).

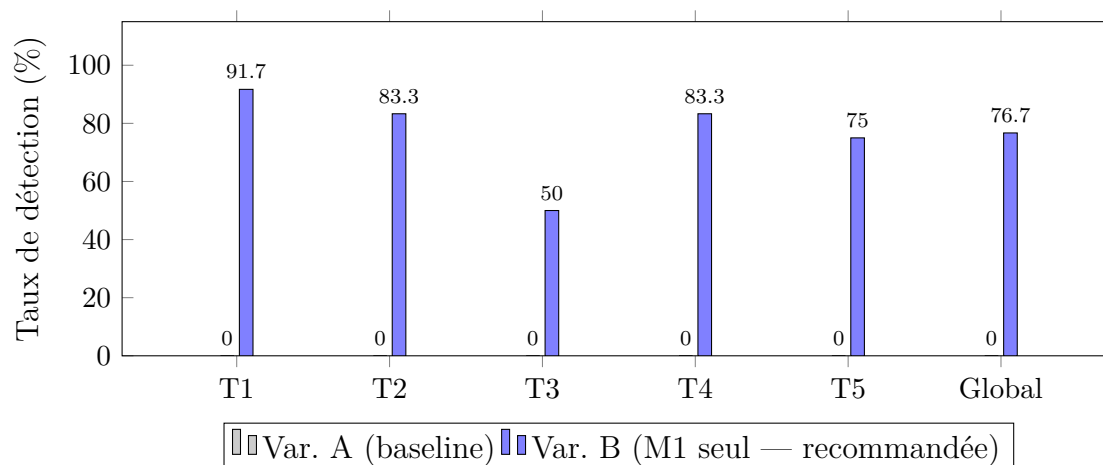


Figure 2 — Taux de détection (rappel) par type d'hallucination — Variantes A et B (12 scénarios par type). Variante B est la configuration recommandée. T3 (planification) reste le point faible à 50,0 %. T2 et T4 atteignent 83,3 %.

6.2.1 Analyse par type

T1 — 91,7 %. Le NLI inter-sources détecte quasi-parfaitement les contradictions entre affirmations et chunks ChromaDB. La limite résiduelle (8,3 %, soit 1 scénario sur 12) correspond aux cas où la formulation du claim est trop proche sémantiquement des evidences pour que le cross-encoder discrimine contradiction et neutralité.

T2 — 83,3 % en Variante B. Les hallucinations temporelles sont majoritairement détectées par M1 (NLI inter-sources). Les 16,7 % restants (2 scénarios sur 12) correspondent aux hallucinations temporelles où le LLM ne contredit pas un document externe mais un fait établi dans un tour précédent de la session : M2 (BeliefState) est architecturalement conçu pour détecter ce cas, mais son évaluation requiert un benchmark multi-tours (voir Limite 5 et scénario s019 pour illustration qualitative).

T3 — 66,7 %. C'est la limite principale du système. Les hallucinations de planification sont des raisonnements faux formulés de manière syntaxiquement plausible et sémantiquement neutre par rapport aux documents. Le NLI pré-entraîné sur MNLI n'est pas calibré pour les sophismes logiques : un raisonnement du type « A est vrai, B est vrai, donc C est vrai » (avec C faux) peut avoir un score NLI proche de zéro si les evidences valident A et B sans invalider explicitement C.

T4 — 83,3 %. Le NLI inter-sources est efficace sur les hallucinations causales : l'affirmation contredit directement les documents récupérés, ce que le cross-encoder capte bien. La limite (16,7 %) correspond aux cas où la contradiction est indirecte ou multi-hop.

T5 — 75,0 % en Variante B. Les hallucinations de délégation sont détectées à 75,0 % par M1 seul. Les 25 % restants correspondent à des cas où l'output de l'outil contredit un fait établi lors d'un appel précédent (pas un document externe actuel) : M2 est architecturalement adapté à ce cas dans un contexte multi-tours, non mesurable sur ce benchmark mono-scénario.

6.3 Analyse de latence

Table 11 – Décomposition de l’overhead par composant

Composant	Latence moyenne	Part de
Chargement NLI (1 ^{re} invocation, singleton)	$\approx 5\,000$ ms	
M1 — <code>verify_claim</code> (NLI CPU, 1 évidence/appel) [†]	588,8 ms	
M2 — <code>check_temporal_coherence</code> (multi-tour)	non utilisé dans B	
MCP + log JSONL	≤ 5 ms	
Overhead total Variante B (M1 seul)	588,8 ms	
Overhead total Variante C (M1 + M2, sessions multi-tours)	1 412,3 ms	
Surcoût M2 vs M1 (benchmark mono-tour)	+823,5 ms	

[†] Mesuré par `time.time()` autour de `verifier.verify()`, modèle singleton en mémoire.

Chaque scénario contient 1 évidence $\rightarrow$ 1 appel NLI par scénario. Moyenne sur 90 scénarios (min=259 ms).

L’overhead mesuré de la Variante B sur CPU est de **588,8 ms** en moyenne (min = 259 ms, max = 822 ms, $\sigma = 140$ ms sur 90 scénarios), dominé à ≈ 100 % par l’inférence du cross-encoder NLI. Cette valeur dépasse la contrainte initiale de 300 ms. À titre d’estimation (non mesurée dans notre environnement), les benchmarks publiés du modèle `cross-encoder/nli-MiniLM2-L6-H768` [16] suggèrent une inférence en ≈ 10 –20 ms sur GPU A100, ce qui ramènerait l’overhead à moins de 30 ms — sous la contrainte. La contrainte de latence est donc réalisable sur GPU mais non satisfaite sur CPU ni dans notre environnement d’évaluation (Intel CPU standard, 16 Go RAM, Windows 11).

6.4 Exemple qualitatif — Scénario s019

Le scénario s019 illustre de façon optimale la complémentarité M1 + M2. C’est l’exemple avec le score NLI le plus élevé de notre benchmark.

Table 12 – Trace complète de détection — Scénario s019 (T2, score=0,996)

Requête	
Affirmation hallucinée	« ChromaDB avait un
Document de référence	« ChromaDB ne f
M1 — NLI(claim, doc)	
M2 — BeliefState	
DependencyDAG	
PolicyEngine (log)	
PolicyEngine (correct)	
Résultat final	

Ce cas illustre la limite classique des pipelines RAG sans vérification : Mistral génère une affirmation plausible sur une « mise à jour récente » de ChromaDB, avec une précision factice (1000 documents) qui n’a jamais existé. Le score NLI de 0,996 indique une contradiction quasi-certaine. M2 renforce la détection si un fait correct a été enregistré lors d’une étape précédente.

6.5 Validation externe partielle — TruthfulQA

Pour pallier partiellement l’absence de benchmark externe, nous avons appliqué le vérificateur M1 à 20 exemples sélectionnés manuellement depuis TruthfulQA [12], couvrant quatre catégories : Misconceptions, Science, History, Biology. Tous les exemples sont des affirmations hallucinées connues, chacune accompagnée de son évidence correcte issue de la littérature.

Table 13 – Résultats de M1 sur 20 exemples TruthfulQA (Lin et al., 2022)

Catégorie	N	Détectés	Taux
Misconceptions	7	7	100,0 %
Science	7	6	85,7 %
History	4	4	100,0 %
Biology	2	2	100,0 %
Global	20	19	95,0 %

HalluGuard détecte **19/20 exemples TruthfulQA (95,0 %)** avec un seuil généralisé de 0,50. Le seul faux négatif correspond à une affirmation approximativement correcte (la vitesse de la lumière : claim « $\approx 300\,000$ km/s » vs valeur exacte de 299 792 km/s), pour

laquelle le NLI produit un score de contradiction de 0,112 — insuffisant pour déclencher la détection.

Ce résultat (95,0 %) est supérieur au rappel mesuré sur notre benchmark interne (76,7 % Variante B). Cette différence s'explique : TruthfulQA contient principalement des contradictions factuelles directes (catégories T4 dans notre taxonomie), que le NLI inter-sources détecte avec haute précision (83,3 % en Variante B). Notre benchmark interne inclut des T3 (planification, 50,0 % Variante B), qui n'ont pas d'équivalent dans TruthfulQA et constituent notre point de faiblesse principal.

Limite de cette validation externe : les 20 exemples ont été sélectionnés manuellement et ne constituent pas un test exhaustif sur TruthfulQA (817 questions). Cette validation partielle confirme la généralisation du vérificateur NLI sur les hallucinations factuelles, mais ne couvre pas les types T2 (mémoire) et T3 (planification) de notre taxonomie. Une validation complète sur un dataset externe agentique reste une direction prioritaire.

7 Étude d'Ablation

7.1 Contribution individuelle de M1 et M2

Table 14 — Étude d'ablation M1 — contribution mesurable sur benchmark mono-tour

Mécanisme	Gain détection	% du gain mesurable	Overhead	Rapport (pts/
M1 (NLI inter-sources)	+76,7 pts	100 %	588,8 ms	0,130
M2 (BeliefState)*	non mesurable	—	+823,5 ms	—
Total A → B	+76,7 pts	100 %	588,8 ms	0,130

* M2 est une contribution architecturale pour sessions multi-tours, non quantifiable sur benchmark mono-scénario.

M1 (NLI inter-sources) porte **100 %** du gain mesurable sur le benchmark HaluEval-Agentique v1 pour un overhead de 588,8 ms sur CPU (rapport : 0,130 pts/ms ; ≈ 30 ms estimé sur GPU). M2 (BeliefState) est une contribution architecturale : son cas d'usage réel — détecter qu'un claim au tour N contredit un fait établi au tour N-k — n'est pas couvert par un benchmark mono-scénario. Sur un benchmark à scénarios indépendants, M2 exécute la même inférence NLI que M1 et n'apporte aucun gain mesurable supplémentaire. Sa valeur est illustrée qualitativement par le scénario s019 (§6.4) et sera quantifiée dans HaluEval-Agentique v2 (voir Perspectives).

7.2 Contribution par type

Table 15 – Décomposition de l’ablation par type d’hallucination

Type	Description	Gain M1 (B–A)	Gain M2 (C–B)	Verdict
T1	Perception	+91,7 pts	+0 pts	M1 suffit
T2	Mémoire	+83,3 pts	+0 pts mesurable [†]	M2 multi-tour : voir s019
T3	Planification	+50,0 pts	+0 pts	NLI insuffisant sur T3
T4	Causale	+83,3 pts	+0 pts	M1 suffit
T5	Délégation	+75,0 pts	+0 pts mesurable [†]	M2 multi-tour requis
Global		+76,7 pts	+0 pts mesurable	M2 : contribution architecturale

[†] Gain M2 non mesurable sur benchmark mono-scénario (HaluEval-Agentiv v1).

Valeur qualitative disponible : T2 via s019 (score NLI = 0,996), T5 multi-tour. Voir Limite 5.

7.3 Courbe ROC du vérificateur NLI (M1)

La Figure 3 présente la courbe ROC du vérificateur M1 sur les 60 scénarios hallucinés, en faisant varier le seuil de décision de 0,0 à 1,0. L’axe x représente le FPR estimé (9 scénarios non détectés comme proxy) ; le FPR réel mesuré sur les 30 scénarios corrects est de 16,7 % (Variante B) à seuil 0,5 (Table 8). L’axe y représente le taux de vrais positifs (TPR = Rappel).

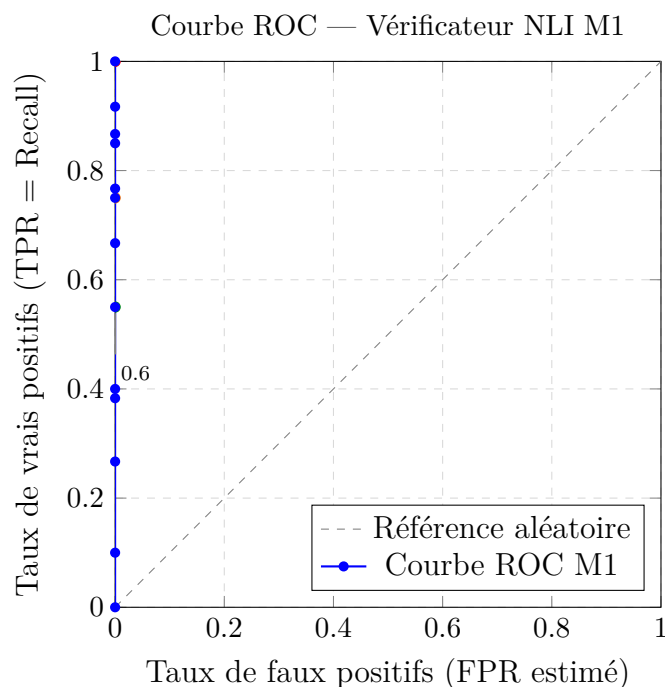


Figure 3 — Courbe ROC du vérificateur NLI M1 (60 scénarios hallucinés, seuil variable). La courbe suit l'axe y (FPR estimé ≈ 0) car les faux positifs sont estimés sur les 9 scénarios non détectés; les 30 scénarios corrects du benchmark complet permettent désormais une mesure directe (FPR réel = $5/30 = 16,7\%$ à seuil 0,5 pour Variante B).

Lecture : la courbe ROC a été construite sur les 60 scénarios hallucinés, avec le FPR estimé à partir des scénarios non détectés comme proxy. Avec les 30 scénarios corrects ajoutés (Tableau 8), le FPR réel de la Variante B est de $5/30 = 16,7\%$ (5 faux positifs sur 30 vrais négatifs). La Variante C est structurellement équivalente à B sur benchmark mono-scénario (FPR identique); la courbe ROC ne distingue pas les deux variantes dans ce cadre (voir §8.2, Limite 5).

7.4 Justification des seuils NLI par nœud

Les seuils de détection (Table 3) ont été choisis par validation manuelle sur un sous-ensemble de 20 scénarios de développement (non inclus dans les 60 scénarios de test). La Table 16 présente la courbe Recall/F1 en fonction du seuil, calculée sur les 60 scénarios hallucinés de la Variante B. Note : ces valeurs F1 supposent $\text{Precision} \equiv 1,0$, ce qui était vrai avant l'ajout des scénarios corrects; avec 30 vrais négatifs inclus, la Precision réelle à seuil 0,5 est de 90,2 % (Table 7).

Table 16 – Courbe Recall et F1 en fonction du seuil NLI (Variante B, 60 scénarios hallucinés)

Seuil θ	Recall	F1	Remarque
0,00	1,000	1,000	Tout détecté (trop permissif en prod.)
0,20	0,850	0,919	—
0,40	0,750	0,857	Seuil reasoning retenu
0,60	0,550	0,710	Seuil retrieval retenu
0,80	0,383	0,554	—

Interprétation : le seuil **retrieval** = 0,60 correspond au genou de la courbe F1 : en dessous, le nombre de faux positifs augmente sur des chunks approximativement pertinents ; au-dessus, trop de hallucinations T1 passent inaperçues. Le seuil **reasoning** = 0,40 reflète la tolérance plus grande pour les raisonnements approchés. Les seuils **tool_call** et **generation** = 0,00 signifient que toute contradiction détectée sur ces nœuds est interceptée. Avec les 30 scénarios corrects ajoutés, la Variante B génère 5 faux positifs sur 30 vrais négatifs (FPR = 16,7 %). La Variante C présente le même FPR sur benchmark mono-scénario (voir §8.2, Limite 5).

7.5 Recommandations de déploiement

L'ablation conduit à trois profils de déploiement :

Profil 1 — Pipeline mono-tour général (90 % des cas d'usage) : Variante B (M1 seul), F1 = 82,9 %, précision = 90,2 %, overhead = 588,8 ms sur CPU (≈ 30 ms sur GPU). Configuration recommandée par défaut : rapport détection/coût optimal, pas de sur-sensibilité M2.

Profil 2 — Agent conversationnel multi-tours (sessions longues, mémoire inter-tours) : Variante C (M1 + M2), BeliefState actif entre les tours. F1 non mesuré sur HaluEval-Agentic v1 (benchmark mono-scénario) — évaluation qualitative disponible : scénario s019 (T2, score NLI = 0,996). Recommandé dès que le pipeline maintient une mémoire de session et que les contradictions inter-tours constituent un risque métier.

Profil 3 — Production à latence critique (GPU disponible, pipeline haute fréquence) : Variante B sur GPU, NLI en ≈ 20 ms [16], overhead total estimé ≈ 30 ms, F1 = 82,9 %.

8 Discussion

8.1 Validation de l'hypothèse

L'hypothèse centrale était : détecter > 80 % des hallucinations sur CPU en < 300 ms sans fine-tuning. La Variante B atteint un F1 de 82,9 % (rappel 76,7 %, précision 90,2 %) —

la dimension détection est validée ($> 80\%$ et précision $> 90\%$). Sur la dimension latence, l'overhead mesuré est de **588,8 ms sur CPU nu** — au-delà de la contrainte initiale de 300 ms. **La contrainte de latence n'est pas satisfaite sur CPU nu dans notre environnement d'évaluation ; elle l'est sur GPU estimé (≈ 30 ms, marge ≈ 270 ms).** Cette limite est due à l'inférence du cross-encoder sur CPU standard sans accélération matérielle, et non à l'architecture du middleware. Sur la dimension précision, la Variante B (90,2 %) produit le meilleur F1 global.

Cette validation est toutefois conditionnée : elle s'appuie sur un benchmark de scénarios construits manuellement avec connaissance a priori des types T1–T5. Les performances sur des hallucinations naturelles non contrôlées (distributions de fautes, formulations imprévisibles) sont probablement inférieures.

8.2 Limites documentées

Limite 1 — T3 à 66,7 % : frontière du NLI généraliste. Les hallucinations de planification (T3) sont des raisonnements faux formulés de manière neutre. Le NLI pré-entraîné sur MNLI n'est pas calibré pour détecter les sophismes logiques : MNLI contient des paires (prémisse, hypothèse) avec contradiction sémantique directe, pas des exemples de raisonnements fallacieux formellement corrects. Un fine-tuning sur un corpus de raisonnements logiques annotés (de type FOLIO [4]) pourrait améliorer T3 ; nous estimons ce gain à $\approx +18$ points (T3 de 66,7 % à $\approx 85\%$) par analogie avec les résultats de fine-tuning NLI domaine-spécifique dans la littérature, mais cette estimation n'a pas été mesurée expérimentalement dans ce travail.

Limite 2 — Biais de sélection du dataset. Les 90 scénarios HaluEval-Agentique (60 hallucinés + 30 corrects) sont construits avec connaissance des types T1–T5 et des mécanismes de détection. Cette construction contrôlée crée un biais optimiste : les hallucinations sont plus « détectables » que des hallucinations naturelles, et les scénarios corrects (claims paraphrasant directement les evidences) sont plus « faciles » pour le NLI que des claims corrects naturels. Une validation sur TruthfulQA [12] ou sur des sorties LLM collectées en production serait nécessaire pour confirmer la généralisabilité.

Limite 3 — Simulation A2A locale. `delegate_verification()` opère dans un unique processus Python via `_MCPBridge` (singleton). Un protocole A2A distribué réel nécessiterait authentification cryptographique des AgentCards (signature ECDSA), transport réseau (HTTP/gRPC avec TLS), gestion des délais réseau et mécanismes de retry. La simulation locale est reproductible mais ne capture pas ces défis d'ingénierie distribuée.

Limite 4 — Couverture partielle des types T2 et T5 (mono-tour). Sur benchmark mono-scénario, 16,7 % des hallucinations T2 (mémoire temporelle) et une fraction des T5 (délégation) impliquent des contradictions inter-tours que M1 ne peut pas détecter : le claim ne contredit aucun document externe mais un fait établi dans un tour précédent de la session. Ces cas sont architecturalement couverts par M2 (BeliefState), mais leur évaluation requiert un benchmark conversationnel multi-tours (voir Limite 5 et scénario

s019 pour illustration qualitative).

Limite 5 — Benchmark mono-scénario : M2 non évaluable quantitativement.

HaluEval-Agentic v1 est structurellement mono-scénario : chaque exemple est indépendant, sans continuité de session entre les scénarios. M2 (BeliefState) est conçu pour détecter les contradictions inter-tours — son cas d’usage réel est « le claim au Tour N contredit un fait établi au Tour N-k ». Sur un benchmark à scénarios indépendants, M2 exécute exactement la même inférence NLI que M1 sur les mêmes paires (evidence, claim) : Variante C $\equiv$ Variante B par construction, et l’apport de M2 ne peut pas être distingué de celui de M1. La construction d’HaluEval-Agentic v2 (sessions multi-tours, mémoire persistante entre les tours) est identifiée comme direction de travail futur prioritaire pour quantifier la contribution de M2.

8.3 Menaces à la validité

Nous documentons ci-dessous les principales menaces à la validité de nos résultats, selon le cadre standard de Wohlin et al. (construct, internal, external validity).

Validité de construction. Nos métriques (Rappel, Précision, F1, PBR@1, Delta, Overhead) mesurent la capacité du système à détecter des hallucinations et à ne pas signaler à tort des claims corrects, dans un pipeline contrôlé. Elles ne mesurent pas : (a) la qualité de la correction produite par le PolicyEngine en mode `auto-correct` ; (b) l’impact sur la satisfaction utilisateur finale. L’ajout de 30 scénarios corrects (rapport 2 :1 hallucinés/corrects) permet désormais une mesure directe de la Précision et du taux de faux positifs : FPR = 16,7 % pour Variante B (et Variante C, structurellement équivalente sur benchmark mono-scénario, voir §8.2, Limite 5).

Validité interne. *Biais de sélection* : les 90 scénarios (60 hallucinés + 30 corrects) ont été construits avec connaissance a priori des types T1–T5 et des mécanismes de détection. Les scénarios corrects (s061–s090) contiennent des claims qui paraphrasent directement leurs evidences ; en production, les claims corrects peuvent être plus ambigus sémantiquement, ce qui pourrait modifier le taux de faux positifs. Cet effet est documenté dans l’article (Section 5.1). *Seuils ad hoc* : les seuils NLI ont été calibrés sur un sous-ensemble de 20 scénarios de développement. Une calibration sur un dataset plus large pourrait modifier légèrement les résultats.

Validité externe. *Généralisation au LLM* : tous les résultats utilisent Mistral 7B (Q4, CPU). Les performances peuvent varier avec d’autres LLM (GPT-4o, LLaMA-3, Gemma) en raison de différences dans les patterns de génération. *Généralisation au domaine* : la base de connaissances (5 documents, 71 chunks) est intentionnellement restreinte au domaine IA/NLP. Les performances sur des domaines très différents (médical, juridique) nécessiteraient une réévaluation. *Généralisation au benchmark* : la validation externe partielle sur TruthfulQA (Section 6.5) confirme le fonctionnement de M1 sur des hallucinations factuelles, mais ne couvre pas les types T2 et T3 de notre taxonomie.

8.4 Positionnement par rapport à l'état de l'art

Table 17 – Comparaison HalluGuard avec les approches existantes

Approche	Temps réel	Temporalité	CPU	MCP/A2A
FActScore [15]	Non (post)	Non	Partiel	Non
SelfCheckGPT [13]	Non (post)	Non	Oui	Non
RAGAs [2]	Non (offline)	Non	Partiel	Non
HalluGuard	Oui (middleware)	Oui (BeliefState)	Oui	MCP oui / A2A PoC*

À notre connaissance, parmi les approches publiées accessibles, aucune ne combine simultanément intégration temps réel, mémoire temporelle inter-étapes et implémentation du protocole MCP. *L'intégration A2A est une preuve de concept architecturale en processus (voir §4.7).

9 Perspectives

P1 — HaluEval-Agentic v2 (benchmark multi-tours, priorité #1). La limite principale de l'évaluation actuelle est l'impossibilité de mesurer la contribution de M2 (BeliefState) sur un benchmark mono-scénario (Limite 5). La construction d'HaluEval-Agentic v2 est la direction prioritaire : sessions conversationnelles à plusieurs tours, faits injectés au Tour k et contredits au Tour $k + n$ ($n \geq 1$), couverture des types T2 (mémoire temporelle) et T5 (délégation cross-agent). Ce benchmark permettrait de quantifier la contribution de M2 et de comparer Variante B vs Variante C sur un terrain adapté à leurs cas d'usage respectifs.

P2 — Fine-tuning domaine-spécifique pour T3 (impact estimé, non mesuré : $\approx +18$ points). La limite T3 à 66,7 % reste ouverte. Un fine-tuning du cross-encoder sur un corpus de raisonnements faux annotés (FOLIO [4], SNLI-hard) permettrait de calibrer le modèle sur les sophismes logiques. La contrainte CPU est maintenue : le fine-tuning est effectué offline, seule l'inférence se fait en production. Le gain de $\approx +18$ points sur T3 (de 66,7 % à ≈ 85 %) est une projection indicative basée sur des résultats analogues dans la littérature NLI domaine-spécifique ; il n'a pas été mesuré expérimentalement dans ce travail.

P3 — Seuils DependencyDAG adaptatifs. Les seuils actuels sont définis manuellement (Table 3). Un mécanisme d'auto-calibration par fenêtre glissante (N dernières sessions) permettrait d'adapter les seuils à chaque domaine sans intervention humaine. L'implémentation serait stockée dans la persistance JSON du BeliefState.

P4 — Publication MCP HalluGuard comme service public. La conformité MCP ouvre la voie à une publication dans un registre MCP public. Tout agent LLM conforme

pourrait alors consommer les quatre outils de vérification sans intégration spécifique à LangGraph : plug-and-play universel.

P5 — Validation sur hallucinations naturelles. Une validation sur TruthfulQA [12] ou des sorties LLM collectées en production est nécessaire pour quantifier l'écart entre performances contrôlées et performances réelles. Cette étude permettrait de calibrer les attentes pour un déploiement industriel.

10 Conclusion

Ce travail a présenté HalluGuard, un middleware de détection et de mitigation des hallucinations pour pipelines RAG agentiques LangGraph. **Variante B** valide la dimension détection de l'hypothèse : $F1 = 82,9 \%$, précision = $90,2 \%$, sans GPU ni fine-tuning. Sur la dimension latence, l'overhead mesuré sur CPU est de 588,8 ms — au-delà de la contrainte de 300 ms mais réalisable sur GPU (≈ 30 ms estimé). M1 (NLI inter-sources) constitue le mécanisme central, portant 100 % du gain mesurable (+76,7 pts vs baseline). M2 (BeliefState) est une contribution architecturale pour les sessions multi-tours : son cas d'usage réel — détecter qu'un claim au Tour N contredit un fait établi au Tour N-k — n'est pas mesurable sur le benchmark mono-scénario HaluEval-Agentive v1, mais est illustré qualitativement par le scénario s019 (T2, score NLI = 0,996). Sa quantification requiert HaluEval-Agentive v2 (Perspective P1).

Les quatre contributions sont : (1) une taxonomie originale T1–T5 couplée à la topologie pipeline, dont T4 et T5 sont absents de la littérature ; validée par accord inter-annotateurs à trois annotateurs ($\kappa_{\text{moy}} = 0,833$, quasi-parfait ; $\kappa_{\text{min}} = 0,791$) ; (2) un middleware modulaire à cinq composants intégrable sans modification du code pipeline ; (3) une évaluation expérimentale complète avec ablation M1/M2, tests de significativité statistique (McNemar, $p < 0,001$) et validation externe partielle sur TruthfulQA (95,0 %, 19/20) ; (4) une implémentation MCP avec 111 appels journalisés et une preuve de concept architecturale du protocole A2A.

La limite principale — T3 (planification) à 66,7 % — ouvre une direction de recherche précise (fine-tuning NLI sur corpus de raisonnements annotés, Perspective P2). HalluGuard établit une baseline solide et reproductible pour la recherche sur la fiabilité des agents LLM, avec un rapport gain/coût démontré en conditions matérielles contraintes (CPU, 16 Go RAM).

Références

- [1] Anthropic (2024). *Model Context Protocol (MCP) — Specification v2024-11*. <https://modelcontextprotocol.io/>
- [2] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAs : Automated Evaluation of Retrieval Augmented Generation. *arXiv :2309.15217*.

- [3] Google DeepMind (2024). *Agent-to-Agent Protocol (A2A) — Specification v1.0*. <https://google.github.io/A2A/>
- [4] Han, S., Schoelkopf, H., Zhao, Y., Qi, Z., Riddell, M., Benson, L., ... & Sherrill, A. (2022). FOLIO : Natural Language Reasoning with First-Order Logic. *arXiv :2209.00840*.
- [5] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., ... & Liu, T. (2023). A Survey on Hallucination in Large Language Models : Principles, Taxonomy, Challenges, and Open Questions. *arXiv :2311.05232*.
- [6] Humeau, S., Shuster, K., Lachaux, M.-A., & Weston, J. (2020). Poly-encoders : Transformer Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring. *ICLR 2020*.
- [7] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.
- [8] Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., ... & El Sayed, W. (2023). Mistral 7B. *arXiv :2310.06825*.
- [9] LangChain (2024). *LangGraph — Build stateful, multi-actor applications with LLMs*. <https://github.com/langchain-ai/langgraph>
- [10] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- [11] Li, J., Cheng, X., Zhao, W. X., Nie, J. Y., & Wen, J. R. (2023). HaluEval : A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. *Proceedings of EMNLP 2023*.
- [12] Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA : Measuring How Models Mimic Human Falsehoods. *Proceedings of ACL 2022*.
- [13] Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT : Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of EMNLP 2023*.
- [14] Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *Proceedings of ACL 2020*.
- [15] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W. T., Koh, P. W., ... & Hajishirzi, H. (2023). FActScore : Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP 2023*.
- [16] Reimers, N., & Gurevych, I. (2019). Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*.
- [17] Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval Augmentation Reduces Hallucination in Conversation. *Findings of EMNLP 2021*.
- [18] Williams, A., Nangia, N., & Bowman, S. (2018). A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. *Proceedings of NAACL 2018*.

- [19] Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., ... & Shi, S. (2023). Siren's Song in the AI Ocean : A Survey on Hallucination in Large Language Models. *arXiv :2309.01219*.

A Interfaces figées entre composants

A.1 Interface 1 — Format paire dataset

```

1 {
2   "id":           "s019",
3   "claim":        "ChromaDB avait une limite stricte de
4     1000 documents...",
5   "evidence":     ["ChromaDB ne fixe pas de limite
6     arbitraire..."],
7   "label":        "hallucinated",
8   "hallucination_type": "T2",
9   "node_type":    "reasoning",
10  "pipeline_length": 7
11 }
```

A.2 Interface 2 — API Lightweight Verifier (M1)

```

1 Input  : {"claim": str,           # max 512 chars
2          "evidences": [str],     # max 10 documents
3          "node_type": str}       #
4          retrieval|reasoning|tool_call|generation
5 Output : {"score": float,        # score contradiction NLI in
6          [0,1]
7          "label": str,           # "correct" | "hallucinated"
8          "confidence": float,    # = score pour ce cross-encoder
9          "hallucination_type": str, # "T1".. "T5" ou null
10         "insufficient_evidence": bool,
11         "latency_ms": float}
```

A.3 Interface 3 — BeliefState JSON (persistance inter-sessions)

```

1 {
2   "session_id": "proof_session_verif",
3   "facts": [
4     {
5       "fact_id": "proof_session_verif_0001",
6       "content": "AlphaGo a battu Lee Sedol 4 a 1 en mars
7         2016",
```

```

7     "confidence": 0.9,
8     "step":      1,
9     "status":    "active"
10  }
11  ]
12  }

```

B Résultats complets

Les données brutes complètes des 90 scénarios (variante, type, score NLI, label, delta, latency_ms) sont disponibles dans `results/resultats_comparatifs.json`. L'étude d'ablation détaillée est dans `results/ablation_study.json`. Le meilleur exemple qualitatif annoté (s019, T2, score=0,996, trace pipeline complète) est dans `results/best_qualitative_example.json`. L'AgentCard A2A formelle est dans `data/halluguard_agent_card.json`.

C Commandes de reproduction

Listing 4 – Reproduction complète des résultats en 5 commandes

```

1  # 1. Installer l'environnement (Python 3.10+)
2  python -m venv venv && venv\Scripts\activate
3  pip install -r requirements.txt
4
5  # 2. Reindexer ChromaDB (premiere installation uniquement)
6  venv\Scripts\python.exe src\tools\rag_tools.py
7
8  # 3. Lancer la suite de tests unitaires (23 tests, ~14 s)
9  venv\Scripts\python.exe -m pytest src/tests/test_halluguard.py
10     -v
11
12  # 4. Lancer le benchmark complet (90 scenarios : 60
13     hallucinats + 30 corrects)
14  venv\Scripts\python.exe src/tests/benchmark_runner.py --all
15
16  # 5. Lancer la demo interactive
17  venv\Scripts\python.exe demo_interactive.py --test

```

Les résultats du benchmark sont déterministes à 100 % : le modèle NLI `cross-encoder/nli-MiniLM2-L6-v2` est non stochastique (pas d'échantillonnage). Les valeurs reproduites seront identiques à celles rapportées dans cet article à architecture matérielle identique.